

8교시: storage/network 통합 실험

수업 목표

- volume과 network를 함께 쓰는 PostgreSQL 실습을 완성한다.
- 데이터 보존과 DNS 접속을 한 번에 검증한다.
- cleanup audit을 수행한다.

강의 전개

Day 2 마지막 교시는 전체를 합친다. PostgreSQL container는 named volume을 사용하고 custom network에 붙는다. client container는 host port 없이 network DNS로 접속한다. container를 교체해도 data가 살아있는지 확인한 뒤 무엇을 지우고 무엇을 남길지 결정한다.

이 교시는 설명만 듣고 지나가지 않는다. 명령은 반드시 code block으로 실행하고, 바로 이어서 검증 명령을 실행한다. 정상 출력이 다를 수 있는 부분은 전체 문자열을 외우지 않고 성공 패턴을 기록한다. 실패도 수업 산출물이다. 실패한 명령, 에러 요약, 가설, 다시 확인한 명령을 함께 남긴다.

실습 명령

```
docker run -d --name paperclip-day2-pg --network paperclip-day2-net -e
POSTGRES_PASSWORD=postgres -v paperclip-pg16-data:/var/lib/postgresql/data
postgres:16
```

```
docker run --rm --network paperclip-day2-net -e PGPASSWORD=postgres
postgres:16 psql -h paperclip-day2-pg -U postgres -d paperclip -c "INSERT INTO
notes(body) VALUES ('day2 integrated evidence'); SELECT * FROM notes;"
```

검증 명령

```
docker stop paperclip-day2-pg
docker rm paperclip-day2-pg
docker run -d --name paperclip-day2-pg-v2 --network paperclip-day2-net -e
POSTGRES_PASSWORD=postgres -v paperclip-pg16-data:/var/lib/postgresql/data
postgres:16
docker run --rm --network paperclip-day2-net -e PGPASSWORD=postgres
```

```
postgres:16 psql -h paperclip-day2-pg-v2 -U postgres -d paperclip -c "SELECT *
FROM notes;"
```

실패 드릴과 오해 교정

상황	해석
SELECT가 실패	table 생성 여부와 volume mount path를 확인한다.
network DNS 실패	client와 DB가 같은 network인지 inspect한다.
cleanup 과잉	volume rm은 Day 2 evidence를 지울 수 있다.

Cleanup

```
docker stop paperclip-day2-pg-v2 || true
docker rm paperclip-day2-pg-v2 || true
docker network rm paperclip-day2-net || true
# volume은 Day 5 Compose에서 재사용할 수 있으므로 기본적으로 남긴다.
# docker volume rm paperclip-pg16-data
```

Cleanup은 비용과 데이터 안전을 동시에 다룬다. container를 지우는 명령과 volume/network/image를 지우는 명령은 의미가 다르다. 특히 volume 삭제는 database data 삭제일 수 있으므로 실습 volume인지 확인한 뒤 실행한다.

Evidence

항목	제출 기준
통합 run	volume+network 사용 command
Data 보존	container 교체 후 SELECT 결과
Cleanup audit	container/network/volume 처리 결정

강의자 설명 포인트

이 실습의 핵심은 명령어 자체가 아니라 경계다. container는 실행 단위이고, volume은 data lifecycle이며, network는 통신 경계다. 학생이 docker run 한 줄을 볼 때 -v, --network, -p를 옵션 목록으로 외우면 뒤에서 Compose와 Kubernetes로 넘어갈 때 같은 혼란이 반복된다. 그래서 각 옵션을 "무엇을 container 밖으로 분리하는가"라는 질문으로 읽게 한다.

강의 중에는 성공 출력보다 실패 출력의 의미를 더 오래 다룬다. port가 열리지 않은 것은 web server 문제가 아닐 수 있고, DB 접속 실패는 password 문제가 아니라 network boundary 문제일 수 있다.

host terminal, container 내부, 같은 Docker network의 client container는 모두 서로 다른 관찰 위치다. 학생이 어디에서 명령을 실행하는지 말로 먼저 설명한 뒤 CLI를 실행하게 한다.

운영 해석

실무에서 database container를 다룰 때 가장 위험한 실수는 cleanup을 단순 정리로만 보는 것이다. container 삭제는 process와 container writable layer를 정리하는 것이고, volume 삭제는 data를 삭제하는 것이다. network 삭제는 통신 경로를 정리하는 것이다. 이 세 가지를 구분하지 않으면 실습은 성공해도 운영 사고를 배운 셈이 된다.

README에는 "실행됐다" 대신 "어떤 data를 남기고 무엇을 삭제했는지"를 써야 한다. Day 2 evidence는 Day 5 Compose에서 volumes와 networks를 읽는 기준이 된다. Compose의 YAML 항목은 갑자기 생긴 문법이 아니라 Day 2에서 손으로 실행한 storage/network 결정을 파일로 옮긴 것이다.

README 기록 예시

```
## Storage/Network Evidence
- Container:
- Volume name:
- Volume target path:
- Network name:
- Host port published? yes/no
- Container DNS check:
- Data survived container replacement? yes/no
- Cleanup decision:
```

다음 연결

Day 3는 image와 Dockerfile로 넘어간다. Day 2는 실행된 container의 data와 network를 다뤘고, Day 3는 그 container의 출발점인 image를 직접 만든다.